

CREATING THE READERMANAGER SAMPLE PROJECT

Content

Setting up a Basic Activity project.....3

Create a scrolling message list.....5

Modify the code in onCreate() to add a responder.....11

Add menu items to Connect/Change & Disconnect the Reader.....12

About TSL®.....16

 About16

 Contact.....16

Setting up a Basic Activity project

IDE: Android Studio v3.2.1

The ReaderManager App demonstrates how to handle connections to TSL Readers using the Android v2.0 SDK. Start from a newly created Basic Activity project as described below using the name “ReaderManager”.

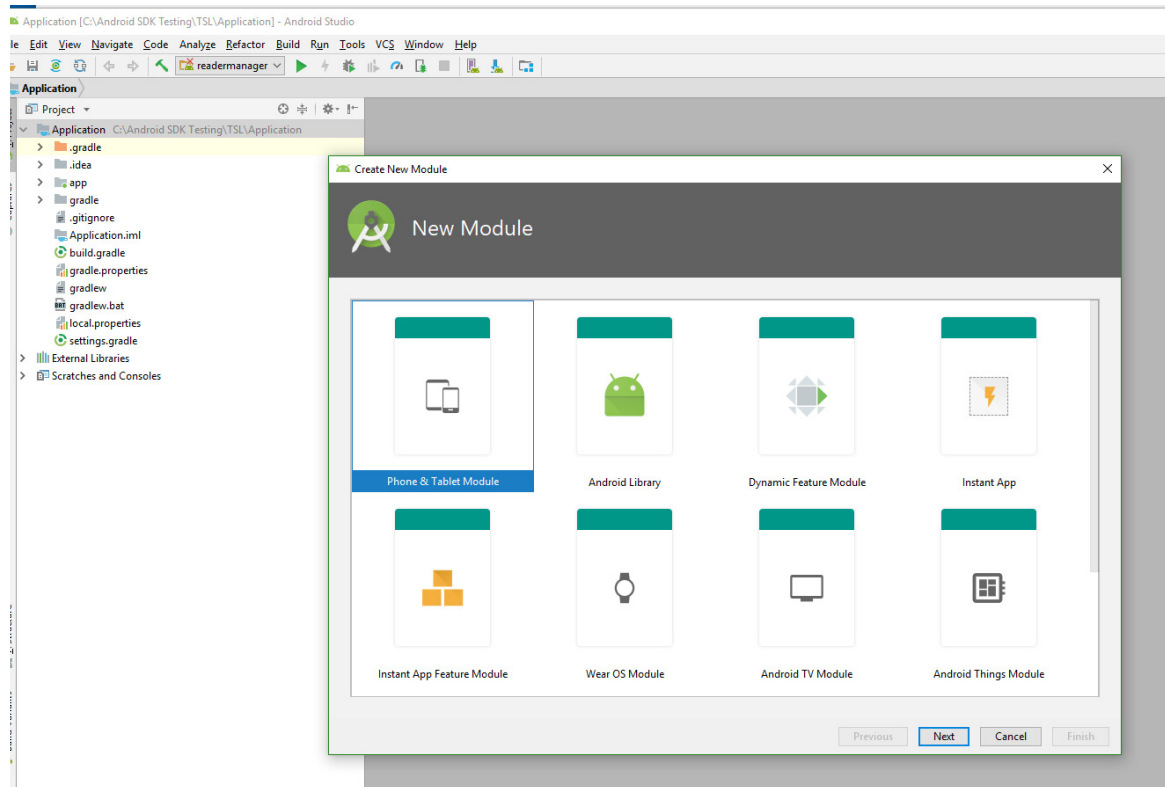


FIGURE 1: Creating a new module

1. In the Project explorer:
2. Change to the Project View
3. Select New->Module
4. Choose Phone & Tablet Module
5. Give the Sample Project a suitable name (No spaces e.g. ReaderManager)
6. Change the package and the package name (the sample uses com.uk.tsl.rfid.samples.readermanager)
7. Leave the Minimum SDK at the default of API15, Android 4.0.3 (IceCreamSandwich)
8. Choose the Basic Activity
9. Rename the Activity to match the project name i.e. ReaderManagerActivity
10. Click Finish - it takes a while and does not show any progress !!!

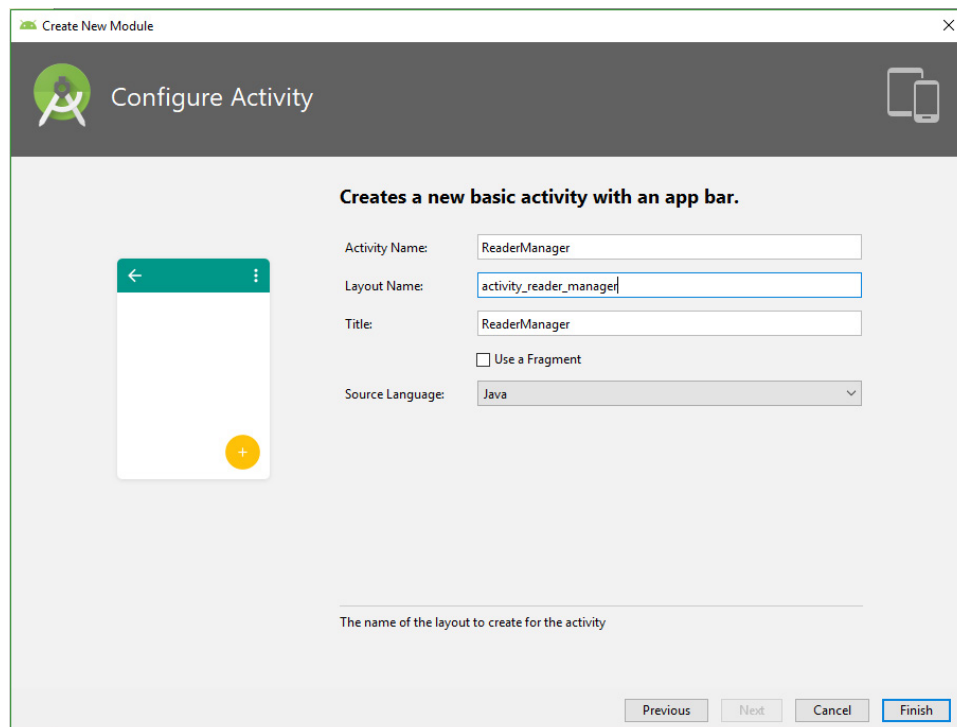


FIGURE 2: Configuring a new basic activity

You are now ready to transform this basic project into the ReaderManager Sample Project.

To help verify that the Reader is connected the App provides a scrolling text display that shows the incoming reader responses.

Create a scrolling message list

This is a simple list that displays messages - it is used to show incoming lines from the Reader and other notifications.

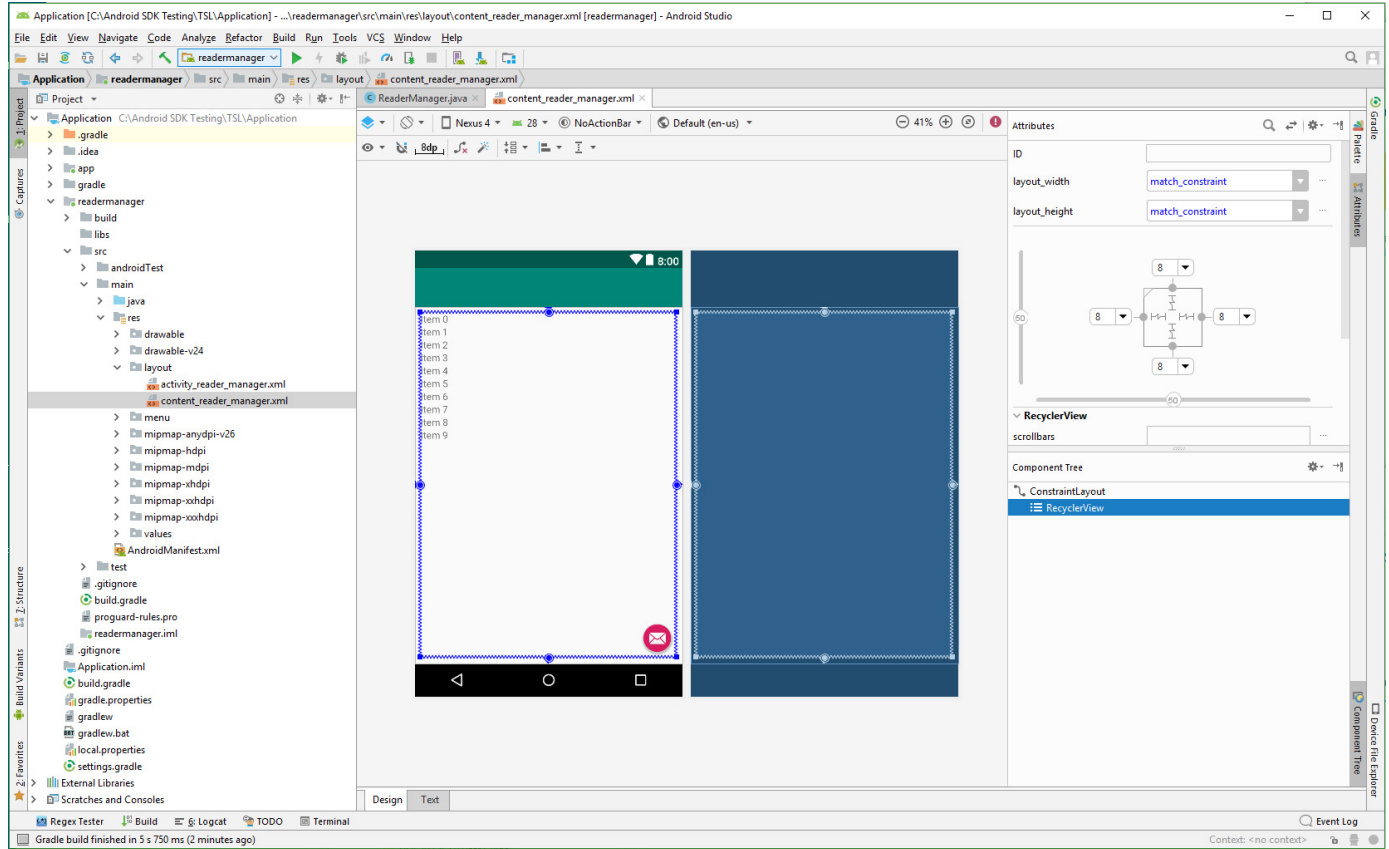


FIGURE 3: Creating RecyclerView

In `content_reader_manager.xml`:

Remove the text view

Add a RecyclerView into the ConstraintLayout

Give it the Id: "message_recycler_view"

Add constraints to keep the control 8dp from each edge

Set the layout width/height to "match_constraint"

Create an xml file in the layout folder called "message_line.xml" with the following contents:

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/messageTextView"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:paddingLeft="4dp"
    android:paddingRight="4dp"
    />
```

Create a new class called MessageViewAdapter - replace the file contents with:

```
package com.uk.tsl.rfid.samples.readermanager;

import android.content.Context;
import android.support.annotation.NonNull;
import android.support.v7.widget.RecyclerView;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import java.util.List;

public class MessageViewAdapter extends RecyclerView.Adapter<MessageViewAdapter.
MessageViewHolder>
{
    private List<String> mMessages;

    // Provide a reference to the views for each data item
    // Complex data items may need more than one view per item, and
    // you provide access to all the views for a data item in a view holder
    class MessageViewHolder extends RecyclerView.ViewHolder
    {
        // each data item is just a string in this case
        TextView mMessageTextView;

        MessageViewHolder(View itemView) {
            super(itemView);

            mMessageTextView = (TextView) itemView.findViewById(R.
id.messageTextView);

            mContext = itemView.getContext();
        }

        private Context mContext;

        void bind(String message)
        {
            mMessageTextView.setText(message);
        }
    }

    // Provide a suitable constructor (depends on the kind of dataset)
    MessageViewAdapter(List<String> readers) {
        mMessages = readers;
    }
}
```

```

        // Create new views (invoked by the layout manager)
        @Override
        @NonNull
        public MessageViewHolder onCreateViewHolder(@NonNull
ViewGroup parent,

        int viewType)
        {
            Context context = parent.getContext();
            LayoutInflater inflater = LayoutInflater.from(context);

            // create a new view
            View readerView = inflater.inflate(R.layout.message_line, parent,
false);

            return new MessageViewHolder(readerView);
        }

        // Replace the contents of a view (invoked by the layout manager)
        @Override
        public void onBindViewHolder(@NonNull MessageViewHolder holder, int
position) {
            // - get element from your dataset at this position
            // - replace the contents of the view with that element
            String message = mMessages.get(position);
            holder.bind(message);
        }

        // Return the size of your dataset (invoked by the layout manager)
        @Override
        public int getItemCount() {
            return mMessages.size();
        }
    }
}

```

In ReaderManagerActivity:

Add fields for the RecyclerView

```

private RecyclerView mRecyclerView;
private MessageViewHolder mAdapter;
private RecyclerView.LayoutManager mLayoutManager;

List<String> mMessages = new ArrayList<>();

```

In onCreate () modify the floating action button code to provide option to clear the list:

```
fab.setOnClickListener(new View.OnClickListener()
{
    @Override
    public void onClick(View view)
    {
        Snackbar.make(view, "Message list", Snackbar.LENGTH_LONG)
            .setAction("Clear", new View.OnClickListener() {
                @Override
                public void onClick(View v)
                {
                    if( mAdapter != null )
                    {
                        mMessages.clear();
                        mAdapter.notifyDataSetChanged();
                    }
                }
            }) .show();
    }
});
```

After the `FloatingActionButton` code, add:

```
// Configure the message list
mRecyclerView = (RecyclerView) findViewById(R.id.message_recycler_view);
mLayoutManager = new LinearLayoutManager(this);
mRecyclerView.setLayoutManager(mLayoutManager);
mAdapter = new MessageViewAdapter(mMessages);
mRecyclerView.setAdapter( mAdapter );
```

Add a method to output messages, ensuring last line is always visible:

```
// Append the given message to the bottom of the message area
private void appendMessage(String message)
{
    final String msg = message;
    mRecyclerView.post(new Runnable() {
        @Override
        public void run() {
            // Select the last row so it will scroll into view...
            mMessages.add(msg);
            final int modifiedIndex = mMessages.size() - 1;
            mAdapter.notifyItemInserted(modifiedIndex);

            mRecyclerView.postDelayed(new Runnable() {
                @Override
                public void run()
```



```
        {
            int listEndIndex = mMessages.size() - 1;
            mRecyclerView.smoothScrollToPosition(listEndIndex);
        }
    }, 20);
}
});
}
```

Add messages for when the Activity pauses and resumes (to aid testing)

Add `onResume()` and `onPause()` methods:

```
@Override
protected void onResume()
{
    super.onResume();

    appendMessage("Resuming...");
}

@Override
protected void onPause()
{
    super.onPause();

    appendMessage("Pausing...");
}
```

Test the app, pause and resume - check that appropriate messages are displayed.

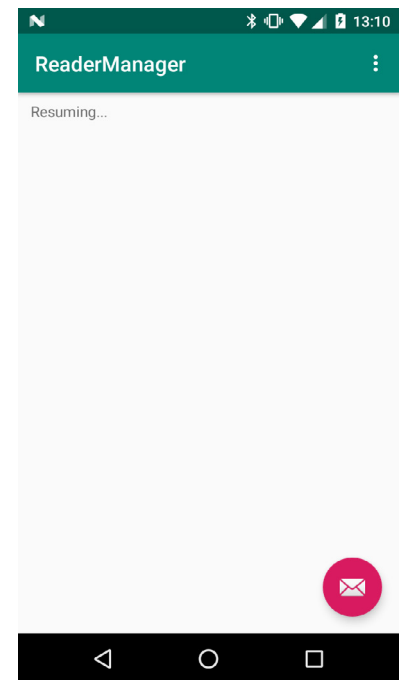
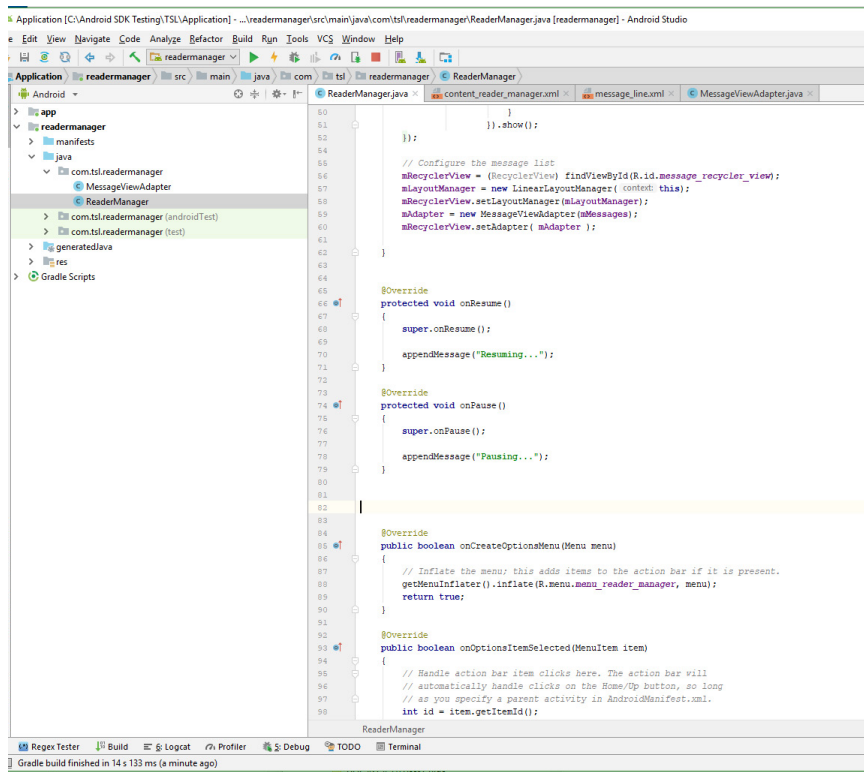


FIGURE 4: Testing the empty App

To add the Rfid.AsciiProtocol-Library to your Android Studio project refer to these instructions:

Using the Rfid.AsciiProtocol-Library with Android Studio.txt

Now use the section “Adding support for TSL ASCII 2 Readers to a project” from the “Getting Started” document to add support for ePop-Loq® Readers.

Use the instructions below rather than the code in the “Add a Select Reader screen” section.

Modify the code in onCreate() to add a responder

```
//
// Add a simple Responder that sends the Reader output to the App message list
//
// Note - This is not the recommended way of receiving Reader input - it is
just a convenient
// way to show that the Reader is connected and communicating - see the other
Sample Projects
// for how to Inventory, Read, Write etc....
//
commander.addResponder(new IAsciiCommandResponder() {
    @Override
    public boolean isResponseFinished() { return false; }

    @Override
    public void clearLastResponse() {}

    @Override
    public boolean processReceivedLine(String fullLine, boolean
moreLinesAvailable)
    {
        appendMessage("> " + fullLine);
        // don't consume the line - allow others to receive it
        return false;
    }
});
```

Add menu items to Connect/Change & Disconnect the Reader

Add a dependency for the DeviceListActivity project to the build.gradle file for ReaderManager:

First add the Strings for the menu text in res/values/strings.xml:

```
<string name="connect_reader_menu_item_text">Connect Reader</string>
<string name="change_reader_menu_item_text">Change Reader</string>
<string name="disconnect_reader_menu_item_text">Disconnect Reader</string>
```

In res/menu/menu_reader_manager.xml add:

```
<item android:id="@+id/connect_reader_menu_item" android:title="@string/connect_
reader_menu_item_text"/>
<item android:id="@+id/disconnect_reader_menu_item" android:title="@string/
disconnect_reader_menu_item_text"/>
```

And hide the item for Settings entry (@+id/action_settings) as it is not used (yet)

```
android:visible="false"
```

In the ReaderManagerActivity.java

Add fields for the menu items:

```
private MenuItem mConnectMenuItem;
private MenuItem mDisconnectMenuItem;
```

In method onCreateOptionsMenu() add:

```
mConnectMenuItem = menu.findItem(R.id.connect_reader_menu_item);
mDisconnectMenuItem= menu.findItem(R.id.disconnect_reader_menu_item);
```

In onPrepareOptionsMenu() enable/disable the menu appropriately:

```
/**
 * Prepare the menu options
 */
@Override
public boolean onPrepareOptionsMenu(Menu menu)
{
    boolean isConnected = getCommander().isConnected();
    mDisconnectMenuItem.setEnabled(isConnected);
}
```

```

        mConnectMenuItem.setEnabled(true);
        mConnectMenuItem.setTitle( (mReader != null && mReader.isConnected() ?
R.string.change_reader_menu_item_text : R.string.connect_reader_menu_item_text));

        return super.onPrepareOptionsMenu(menu);
    }

```

In method `onOptionsItemSelected()` replace `if(){}` for settings with::

```

switch (id) {

    case R.id.action_settings:
        // Handle settings here...
        return true;

    case R.id.connect_reader_menu_item:
        // Launch the DeviceListActivity to see available Readers
        mIsSelectingReader = true;
        int index = -1;
        if( mReader != null )
        {
            index = ReaderManager.sharedInstance().getReaderList().list().
indexOf(mReader);
        }
        Intent selectIntent = new Intent(this, DeviceListActivity.class);
        if( index >= 0 )
        {
            selectIntent.putExtra(EXTRA_DEVICE_INDEX, index);
        }
        startActivityForResult(selectIntent, DeviceListActivity.SELECT_
DEVICE_REQUEST);
        return true;

    case R.id.disconnect_reader_menu_item:
        if( mReader != null )
        {
            mReader.disconnect();
            mReader = null;
        }

        return true;
}

```

Add a handler for the result of the DeviceList Activity:

```
//
// Handle Intent results
//
public void onActivityResult(int requestCode, int resultCode, Intent data) {

    switch (requestCode)
    {
        case DeviceListActivity.SELECT_DEVICE_REQUEST:
            // When DeviceListActivity returns with a device to connect
            if (resultCode == Activity.RESULT_OK)
            {
                int readerIndex = data.getExtras().getInt(EXTRA_DEVICE_INDEX);
                Reader chosenReader = ReaderManager.sharedInstance().
getReaderList().list().get(readerIndex);

                int action = data.getExtras().getInt(EXTRA_DEVICE_ACTION);

                // If already connected to a different reader then disconnect it
                if( mReader != null )
                {
                    if( action == DeviceListActivity.DEVICE_CHANGE || action ==
DeviceListActivity.DEVICE_DISCONNECT)
                    {
                        appendMessage("Disconnecting from: " + mReader.
getDisplayNames());

                        mReader.disconnect();
                        if(action == DeviceListActivity.DEVICE_DISCONNECT)
                        {
                            mReader = null;
                        }
                    }
                }

                // Use the Reader found
                if( action == DeviceListActivity.DEVICE_CHANGE || action ==
DeviceListActivity.DEVICE_CONNECT)
                {
                    mReader = chosenReader;
                    appendMessage("Selected: " + mReader.getDisplayNames());
                    getCommander().setReader(mReader);
                }
            }
            break;
    }
}
```

Additional information can be found in the “Using the Device List Activity” document.

Build and test the App - you now have an App that can connect to TSL ASCII 2 Readers over USB or *Bluetooth®*.

Connect a Reader - when the connection is established you should see:

- the ASCII 2 Reader responses to a “.vr” (which is from the AsciiCommander determining the Reader’s DeviceProperties)
- the “.bl” response from the BatteryStatusCommand issued after the Reader has connected

Now pull and hold the trigger to see the responses to the “.iv” inventory command that is the default single press trigger action.

You now have a basic App that connects to a TSL ASCII 2 Reader - check out the other Sample Projects to see how to use TSL API commands to add RFID functionality to your App.

About TSL®

About

TSL® | Global Leaders in Mobile RFID

Technology Solutions UK Ltd (TSL®) is a leading manufacturer of high performance mobile RFID readers used to identify and track products, assets, data or personnel.

For over two decades, TSL® has delivered innovative data capture solutions to Fortune 500 companies around the world using a global network of distributors and system integrators. Specialist in-house teams design all aspects of the finished products and software ecosystems, including electronics, firmware, application development tools, RF design and injection mould tooling.

TSL® is an ISO 9001:2015 certified company.



ISO 9001: 2015

Contact

Address:	Technology Solutions (UK) Ltd, Suite A, Loughborough Technology Centre, Epinal Way, Loughborough, Leicestershire, LE11 3GE, United Kingdom.
Telephone:	+44 1509 238248
Fax:	+44 1509 214144
Email:	enquiries@tsl.com
Website:	www.tsl.com